

wife-system · 阶段 0 ~ 3 实现后教材

P0

最小 Agent、HTTP 与微信桥接

固定代码基线 6e89762
中文学习版 · 虚拟数据示例

从实际代码、数据流和测试证据出发；每册可独立阅读。

目录

第一册：P0 最小 Agent、HTTP 与微信桥接	3
学习目标与前置知识	3
1. 最小 Agent：模型负责选择，程序负责执行	4
1.1 主数据流	4
小练习	4
2. FastAPI 探针：先证明链路，再连接业务	5
小练习	5
3. Python HTTP 与 TypeScript OpenClaw 桥接	6
小练习	6
4. 微信、桌面端、DeepSeek 与证据边界	7
测试证据分层	7
常见误区	8
阶段练习	9
检查题	10

第一册：P0 最小 Agent、HTTP 与微信桥接

代码基线：**6e89762**。本册只描述提交中存在的实现和已有验收证据。

学习目标与前置知识

学完本册，你应能沿着一次“查询本月虚拟预算”的请求，解释模型何时说话、工具何时执行、HTTP 服务怎样收发 JSON、OpenClaw 怎样把微信入口接到 Python，以及进程内幂等能防住什么。

前置知识：Python 函数、类、异常、字典；了解 TypeScript 语法更好，但不是必需。

首次术语：

- **HTTP**：客户端与服务端交换请求和响应的协议；方法、路径、状态码、请求头和正文共同表达一次调用。
- **JSON**：HTTP 中常用的文本数据格式，由对象、数组、字符串、数字、布尔和空值组成。
- **DTO**（Data Transfer Object）：专门描述边界输入输出形状的数据对象。本项目主要用 Pydantic 模型实现。
- **幂等**：同一个业务请求重复到达，最终业务效果仍与执行一次相同；若同一个键带了不同载荷，应拒绝而不是猜测。
- **Agent tool loop**：模型返回工具调用，程序校验并执行工具，再把工具结果交回模型，直到得到最终回答或触发停止条件。

1. 最小 Agent：模型负责选择，程序负责执行

1.1 主数据流

用户文本

- AgentRunner 调用 ModelProvider
- 模型返回 ToolCall(name, arguments)
- ToolRegistry 查找工具
- Pydantic 校验 arguments
- Python 函数执行
- 工具结果作为 tool message 回到模型
- 模型返回最终中文回答

AgentRunner 在 **agent/loop.py** 第 26 ~ 32 行 固定每次最多 4 轮模型调用、8 次工具调用、1 次写工具和 15 秒 provider 超时。循环主体在第 93 ~ 316 行：它处理模型异常、未知工具、重复工具调用、参数错误、工具异常、暂停和最终文本。

模型边界由 **ModelProvider** 协议 定义。测试可用 **ScriptedModelProvider**，离线演示可用

DeterministicBudgetProvider；真实 DeepSeek 适配器从第 145 行开始，只负责把中立消息转换成 OpenAI 兼容请求并解析响应。

工具本身由 **Tool** 描述：名称、说明、Pydantic 参数模型和处理函数。**Tool.invoke()** 在第 42 ~ 51 行先 **model_validate**，再调用 handler。**ToolRegistry** 在第 54 ~ 86 行负责唯一注册、Schema 列表和按名查找。P0 的 **query_budget** 使用固定虚拟数据，定义在第 89 ~ 139 行。

输入示例：

```
{"role": "user", "content": "这个月还能花多少?"}
```

中间工具调用：

```
{"id": "call-1", "name": "query_budget", "arguments": {"period": "current_month"}}
```

工具输出只含虚拟整数分，最终由模型组织为中文。**ToolCall**、**AssistantTurn** 和 **AgentRunResult** 都继承 Pydantic **BaseModel**，见 **agent/types.py** 第 12 ~ 84 行。**Field(min_length=1)** 让空调用编号和空工具名在进入循环前就失败；**ConfigDict(extra="forbid")** 拒绝未知字段。

边界：模型可以提出调用，但不能绕过注册表直接执行函数；工具参数不是可信身份来源。P0 进程内请求缓存见 **loop.py** 第 48 ~ 91 行，重启或多 worker 后不会共享。

未采用方案：P0 没有引入 LangGraph。一个显式循环更容易逐分支测试；当未来出现多个长时间暂停点和复杂分支时再评估图式编排。

小练习

给 **QueryBudgetArguments** 的 **period** 传入 **next_year**，先预测 Pydantic 会在哪一层拒绝，再在测试中验证。不要改真实数据。

2. FastAPI 探针：先证明链路，再连接业务

探针是一个小而可观察的请求，用于证明服务活着、输入校验有效、响应能往返。它不是财务记账。

`GET /healthz` 在 `api/app.py` 第 207 ~ 209 行返回健康状态。`POST /api/v1/probes` 在第 211 ~ 260 行接收请求头中的幂等键和 DTO，调用 `ProbeService`，把冲突映射为 HTTP 409。

请求 DTO 在 `api/schemas.py` 第 10 ~ 30 行限定 challenge 和响应五个字段。FastAPI 负责路由和 HTTP，Pydantic 负责形状与字段校验，`ProbeService` 负责业务规则，存储对象负责并发安全的进程内记录。

```
// POST /api/v1/probes
// Idempotency-Key: virtual-demo-001
{"challenge":"hello-probe"}

{
  "request_id":"虚拟 UUID",
  "challenge":"hello-probe",
  "receipt":"POC-随机十六进制",
  "created_at":"2026-09-19T20:00:00+08:00",
  "replayed":false
}
```

随机回执由 `ProbeService.new_receipt` 使用 `secrets.token_hex(12)` 生成。时间必须带时区，见第 77 ~ 97 行。
`InMemoryProbeStore.get_or_create()` 在第 45 ~ 60 行持锁检查：同键同 challenge 返回旧记录并标记 `replayed=true`；同键不同 challenge 抛冲突。

这说明幂等不是“每次都返回成功”，而是：

重试情况	结果
同键、同载荷	重放原 request ID、receipt 和时间
同键、不同载荷	409 冲突
新键	创建新记录
Python 重启	P0 内存记录消失，这是已知边界

小练习

画出两个线程同时提交同键同载荷时，锁内的读取、创建与返回顺序。指出如果没有锁，可能产生哪两条不同 receipt。

3. Python HTTP 与 TypeScript OpenClaw 桥接

OpenClaw 插件不是第二套业务系统。它把宿主里的命令或 Agent 工具转换成固定 HTTP 请求，然后严格验证 Python 响应。

```

微信/桌面消息
→ OpenClaw 命令 finance-probe 或工具 finance_probe
→ TypeScript FinanceProbeClient
→ HTTP 127.0.0.1:8000/api/v1/probes
→ Python FastAPI → ProbeService
← 严格五字段 JSON
← TypeScript 映射为安全结果

```

client.ts 第 105 ~ 147 行只允许回环地址并校验超时和输入；**createProbe()** 在第 185 ~ 225 行发送 challenge 与调用方给出的幂等键；底层请求在第 227 ~ 289 行设置超时、禁止重定向、解析 JSON 并映射状态。响应解析在第 45 ~ 95 行要求字段恰好匹配，避免把坏后端响应当成功。

插件在 **index.ts** 第 72 ~ 87 行注册需认证的 **finance-probe** 命令，每次命令调用生成新 UUID；在第 89 ~ 117 行注册 **finance_probe** 工具，并把宿主 **toolCallId** 用作幂等键。TypeScript 不做隐藏重试，调用方能清楚决定是否重试。

常见误区：命令生成的 invocation UUID 不是微信原始事件 ID，所以不能宣称防住微信事件重投；**toolCallId** 也只在宿主重用同一 ID 时提供重放语义。

小练习

分别写出 HTTP 422、409、503 和超时时，桥接层应告诉调用方“修正输入”“更换冲突键”还是“可以重试”，并解释原因。

4. 微信、桌面端、DeepSeek 与证据边界

微信是手机消息入口，Windows 桌面端是另一入口。二者可以把请求送进同一个 Python 后端，后端才持有 Agent、工具和数据规则。入口不同不应复制账本逻辑。

DeepSeek 的职责是理解自然语言、选择工具并组织回答。确定性工具负责金额计算、权限判断和未来的数据库写入。没有 API key 时仍可运行：FastAPI 健康检查、随机探针、虚拟预算工具、脚本化/离线 provider、桥接的本地契约测试；不能声称完成真实 DeepSeek 语言理解。

测试证据分层

证据	实际证明	不能推出
P0 自测	实现者预期的主路径与回归	独立视角、真实微信
C2 独立测试	Node 44 项、执行方 27 项、类型检查/构建、Python 回归和隔离 OpenClaw runtime 通过	微信插件真实账号长期稳定
W1 真实联调	真实微信命令探针、第二次新探针、停服安全失败、自然语言工具路径取得证据	未执行的提醒、恢复、更多微信案例

独立报告见 [phase-0-c2-b2b-report.md](#)，真实联调边界见 [phase-0-w1-wechat-report.md](#)。

常见误区

1. 把 Agent 当成“模型直接运行 Python”。实际是程序读取结构化工具调用后决定是否执行。
2. 把 Pydantic 当数据库。它只校验边界数据，不提供持久化。
3. 把随机 request ID 当幂等键。幂等键必须来自同一业务请求的稳定标识。
4. 把健康检查通过当业务正确。健康检查只证明服务能响应。
5. 把本机回环桥接通过外推到真实微信所有场景。

阶段练习

用离线 provider 完成一次虚拟预算查询，记录每一轮 **ConversationMessage**：用户消息、assistant 工具调用、tool 结果、最终回答。再回答：哪一步即使模型输出错误工具名也不会执行未知函数？

检查题

1. **ToolRegistry** 和 Pydantic 参数模型分别挡住哪类错误？
2. 为什么同一幂等键带不同 challenge 必须冲突？
3. Python 服务重启后，P0 探针的幂等记录为什么消失？
4. OpenClaw 命令 UUID 与微信来源事件 ID 有何不同？
5. 没有 DeepSeek API key 时，哪些链路仍能获得确定性证据？